

Algoritmos y Estructuras de Datos III

Instructores: Diego Ihara, Marcos Villagra
Fecha: 2-jun-2023

Parcial #2

1. Backtracking. Tenemos tres números enteros no negativos x , y y z que satisfacen las siguientes tres relaciones:

$$\begin{aligned}x + y + z &= A \\xyz &= B \\x^2 + y^2 + z^2 &= C\end{aligned}$$

1. Escribe un algoritmo que utilice *backtracking* recursivo y que retorne los valores de x, y, z para valores A, B, C dados como entrada. Los valores de A, B, C son enteros mayores o iguales a 1. Las variables x, y, z solo pueden tomar valores enteros mayores o iguales a 0.
2. Escribe una explicación del algoritmo presentado justificando su funcionamiento.
3. Realiza un análisis de la complejidad temporal de tu algoritmo y presenta una justificación para esa complejidad.

Rúbrica

Criterio	0 Puntos	1 Punto	2 Puntos
Algoritmo	No realiza el ejercicio o el pseudocódigo presentado no intenta resolver el problema	Presenta un pseudocódigo basado en backtracking recursivo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo basado en backtracking recursivo que retorna el resultado correcto en todas las entradas
Correctitud	No presenta ninguna explicación del funcionamiento de su algoritmo o no hay algoritmo	Se observa una explicación correcta del algoritmo presentado justificando su funcionamiento	
Complejidad	No presenta ninguna complejidad o la complejidad es incorrecta	La complejidad asintótica es correcta pero no hay justificación	La complejidad asintótica es correcta y presenta una justificación correcta de la complejidad

Obs. Los criterios de «Correctitud» y «Complejidad» solo serán considerados si el criterio de «Algoritmo» tiene 1 o más puntos.

Respuesta. La idea para un algoritmo de backtracking consiste en observar que las variables x, y, z son enteros no negativos que suman A . Entonces, podemos hacer una búsqueda exhaustiva para todas las combinaciones que suman A , y luego verificamos si se cumplen las otras dos ecuaciones.

//Subrutina de búsqueda de sumas de A

SumaA(valores=[])

begin

if length(valores)==3 and suma(valores) == A and valores[0]*valores[1]*valores[2] == B
and valores[0]**2+valores[1]**2+valores[2]**2 == C then

Return 1

else

Return 0

end-if

```

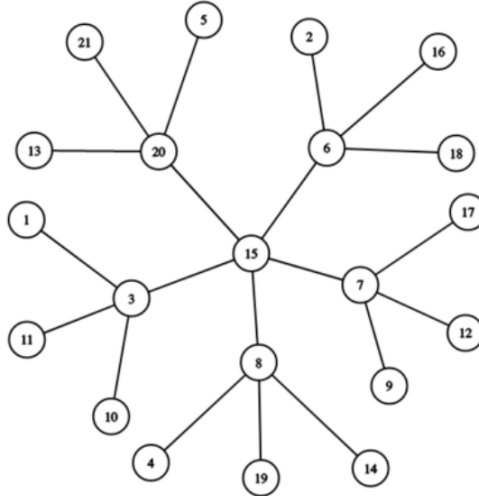
for i=0 to A do
    valores.append(i)
    retorno = SumaA(valores)
    if retorno == 1 then
        Return 1
    else
        valores.pop()
    end-if
end-for
Return 0
end

```

El algoritmo prueba todos los valores de x, y, z que sumen A y luego verifica que cumplan con las otras dos ecuaciones. Entonces, cada variable puede tomar $A + 1$ valores y como nuestro árbol de recursión tiene altura tres, tenemos que la complejidad del algoritmo es $O(A^3)$.

2. Grafos. Un grafo se dice que es un *copo de nieve* si tiene la siguiente estructura. Dados dos números enteros positivos x e y mayores a 1, construimos un copo de nieve como sigue: (i) comenzamos con un vértice central; (ii) conecta al vértice central x vértices nuevos; (iii) conecta y vértices a cada uno de los x vértices conectados al vértice central.

Por ejemplo, para $x = 5$ e $y = 3$ obtenemos el grafo de abajo.



1. Dados x e y ¿Cuántos vértices tiene un copo de nieve? ¿Cuántas aristas tiene? Escribe tu respuesta en términos de x e y .
2. Escribe un pseudocódigo que toma como entrada un copo de nieve y retorna como resultado los valores de x e y . Suponemos que el grafo que se da como input es siempre un copo de nieve; es decir, no necesitas verificar si el grafo es un copo de nieve.
3. Escribe una explicación del algoritmo presentado justificando su funcionamiento.
4. Realiza un análisis de la complejidad temporal de tu algoritmo y presenta una justificación para esa complejidad.

Rúbrica

Criterio	0 Puntos	1 Punto	2 Puntos
Vértices y aristas	No realiza el ejercicio o es incorrecto	Escribe el número correcto de vértices o aristas	Escribe el número correcto de vértices y aristas
Algoritmo	No realiza el ejercicio o el pseudocódigo presentado no intenta resolver el problema	Presenta un pseudocódigo que retorna el resultado correcto en la mayoría de las entradas	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas
Correctitud	No presenta ninguna explicación del funcionamiento de su algoritmo o no hay algoritmo	Se observa una explicación correcta del algoritmo presentado justificando su funcionamiento	
Complejidad	No presenta ninguna complejidad o la complejidad es incorrecta	La complejidad asintótica es correcta pero no hay justificación	La complejidad asintótica es correcta y presenta una justificación correcta de la complejidad

Obs. Los criterios de «Correctitud» y «Complejidad» solo serán considerados si el criterio de «Algoritmo» tiene 1 o más puntos.

Respuesta. Dados x, y , y siguiendo la construcción de un copo de nieve, tenemos en total $xy + x + 1$ vértices. El centro cuenta como un vértice, y el centro tiene x vecinos, y cada vecino del centro tiene a su vez y vecinos.

Para escribir un algoritmo notamos primero que un copo de nieve es un árbol, y por lo tanto, todos los caminos en un copo de nieve son únicos. La idea para calcular x e y consiste primero en identificar dos hojas y encontrar un camino entre estas hojas. Si el camino es de longitud 4, entonces necesariamente tiene que pasar por el centro del copo de nieve, porque un copo de nieve es un árbol. Entonces, el centro del copo de nieve debe de estar siempre en la mitad de un camino de longitud 4 entre una hoja y otra hoja. Una vez que obtenemos el centro del copo de nieve, tenemos que x es el grado del centro e y es el grado de cualquiera de los vecinos del centro. El pseudocódigo de abajo implementa esta idea. La subrutina `findCenter` encuentra el centro de un copo de nieve y la subrutina `findXY` retorna x e y .

```
//Subrutina de búsqueda del centro de un copo de nieve
findCenter(G)
begin
  for all u in G(V) do
    if deg(u) == 1 then
      for all v in G(V) do
        if deg(v) == 1 then
          path = Find path between (u,v) in G
          If path is of length 4 then
            Return the vertex in the center of path
          end-if
        end-if
      end-for
    end-for
  end-for
end

//Subrutina de retorno de x e y
findXY(G)
begin
  center = findCenter(G)
  x = deg(center)
  Let u be adjacent to center in G
  y = deg(u)
  Return x,y
end
```

En nuestro algoritmo de arriba supusimos que ya tenemos los grados de los vértices. De todas formas, si los grados de los vértices no están, podemos correr un DFS o BFS primero para calcular los grados en tiempo lineal. Entonces, la complejidad del algoritmo es dependiente solo de la subrutina `findCenter`. Si n es el número de vértices, entonces `findCenter` corre en tiempo $O(n^2) \cdot O(\text{path finding})$, donde $O(\text{path finding})$ es la complejidad de encontrar un camino entre u y v . Para encontrar un camino entre dos vértices basta con correr un algoritmo DFS o BFS en tiempo lineal. Entonces, la complejidad total del algoritmo es $O(n(n+m))$, donde m es el número de aristas de G .

3. Programación Dinámica Suponga que para subir una escalera se puede avanzar de a 1, 2 o 3 escalones por vez. El objetivo es, dado un número entero n de escalones, retornar la cantidad de formas en las que se puede subir la escalera.

1. Escribe la función de recursión para el problema de subir escaleras. Indica el caso base.
2. Dibuja el árbol de llamadas recursivas para una escalera de 5 escalones.
3. Escribe el costo asintótico $O()$ si se implementa la versión recursiva del algoritmo.
4. Escribe qué estructura de datos se podría utilizar para aplicar la técnica de *memoization* y su costo $O()$ en espacio.
5. Escribe un pseudocódigo implementando el algoritmo de programación dinámica para el problema de la escalera utilizando el enfoque *bottom-up*.

Rúbrica

Criterio	0 Puntos	1 Punto	2 Puntos	3 Puntos
Función	No presenta ninguna función o la misma no es correcta.	Función correcta pero no indica el caso base.	Función correcta e indica el caso base.	
Dibujo	No presenta ningún dibujo, o es incorrecto.	Presenta un dibujo correcto.		
Complejidad	No especifica claramente ninguna complejidad.	Complejidad correcta.		
<i>Memoization</i>	La estructura de datos no es la correcta o es utilizada de manera incorrecta. En este caso el costo asintótico de espacio no tiene valor.	Estructura de datos correcta.	Estructura de datos correcta y costo en espacio correctamente especificados.	
Algoritmo	No presenta ningún pseudocódigo o el pseudocódigo no intenta resolver el problema o el pseudocódigo funciona para una minoría ($\leq$ %50) de las entradas.	Presenta un pseudocódigo que retorna el resultado correcto en una mayoría ($>$ %50) de las entradas. Se considera aunque no sea <i>bottom-up</i> pero sí debe aplicar alguna técnica de programación dinámica.	Presenta un pseudocódigo que retorna el resultado correcto en todas las entradas y cumple con la técnica solicitada.	

Respuesta.

1. Función recursiva.

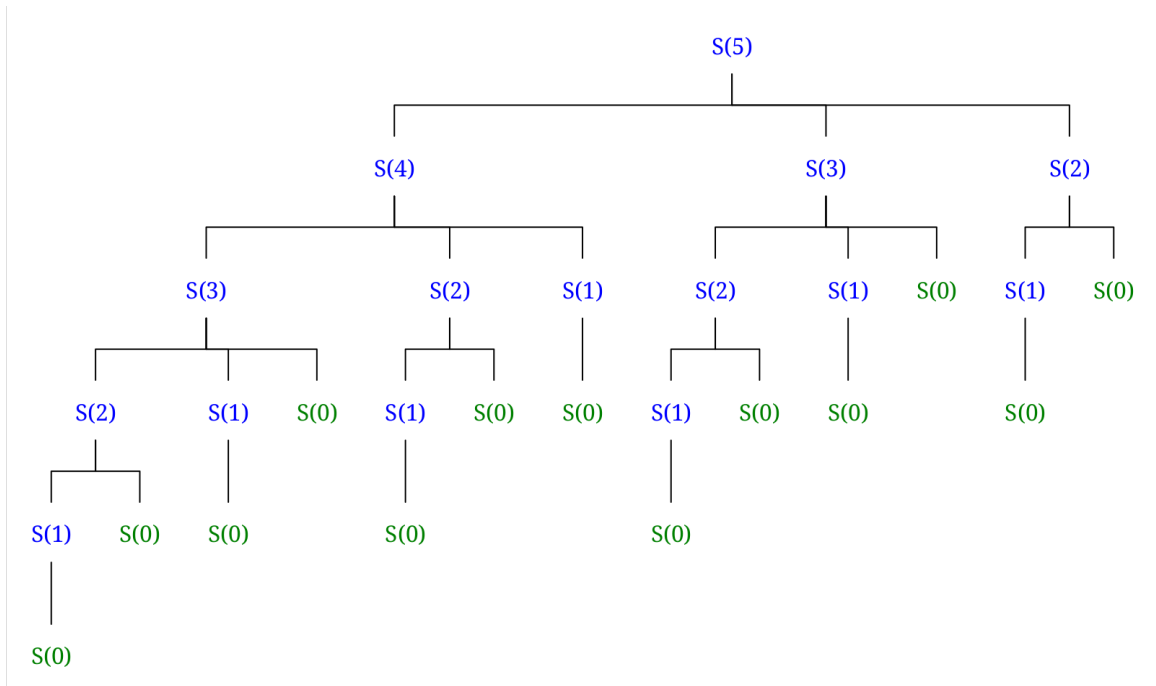
Sea **escalera**(n) una función que retorna el número de formas de subir una escalera de n escalones

$$\text{escalera}(n) = \text{escalera}(n-1) + \text{escalera}(n-2) + \text{escalera}(n-3)$$

Caso base

$$\text{escalera}(0) = 1$$

2. Arbol recursivo:



3. Costo: $O(3^n)$.

4. *Memoization*:

- Se puede utilizar un arreglo de longitud n donde almacenar el resultado de cada llamada a $S()$ para cada valor en $[1 \dots n]$. Costo en espacio $O(n)$.
- Como el valor de n depende de $(n - 1)$, $(n - 2)$ y $(n - 3)$, se puede optimizar a espacio $O(1)$.

5. Ejemplo de pseudocódigo:

```

def S(n):
    if n < 0:
        return 0

    memo = [0 for i in range(n + 1)]
    memo[0] = 1
    memo[1] = 1

    for i in range(2, n + 1):
        memo[i] = memo[i-1] + memo[i-2] + memo[i-3]

    return memo[n]

```